

# INFORME FUNCIONAMIENTO SUGARBAY (BASADO EN CODIGO REAL)

Fecha de revision: 24 de abril de 2026 Repositorio revisado: `Sugarbayweb` (workspace local actual)

---

## 1. Introduccion general

---

Sugarbay es una aplicacion web full-stack para una banda de musica. Combina dos partes principales:

1. Parte editorial/publica: conciertos, noticias, bio, musica y media.
2. Parte e-commerce: tienda, carrito, checkout, pago y cuenta de usuario.

El proyecto esta hecho con Next.js App Router y TypeScript. A nivel de datos usa Prisma sobre PostgreSQL (Neon). El pago real esta conectado con Stripe (Checkout hospedado). ImageKit se usa para servir imagenes de productos y contenido multimedia.

Tecnologias y papel en esta app:

1. Next.js App Router: define rutas por carpetas en `app/` y separa Server Components y Client Components.
2. TypeScript: tipado estricto ( `strict: true` en `tsconfig.json` ) para reducir errores de integracion.
3. Tailwind CSS v4: estilos globales y utilidades, con un sistema visual custom en `app/globals.css` .
4. Prisma: ORM que modela toda la base de datos en `prisma/schema.prisma` y ejecuta consultas desde `lib/repositories/*` .
5. Neon PostgreSQL: base de datos remota conectada por `DATABASE_URL` y `DIRECT_URL` .
6. Stripe: flujo real de pago por tarjeta en checkout, con webhook y sincronizacion de estado de pedido.

7. ImageKit: resolucion de URLs de imagen y endpoint de autenticacion server-side para futuras subidas.

---

## 2. Estructura general del proyecto

---

Carpetas y archivos clave:

1. `app/` : rutas de la web (paginas), layouts globales, estados `loading` y `error`, y APIs ( `app/api/*` ).
2. `components/` : componentes reutilizables por dominio ( `layout`, `auth`, `cart`, `checkout`, `store`, `concerts`, `band`, `music`, `media`, `ui` ).
3. `lib/` : logica de negocio y acceso a datos.
4. `lib/repositories/*` : capa principal de consultas/escrituras Prisma.
5. `lib/auth/*` : sesion JWT en cookie httpOnly + DAL de usuario.
6. `lib/services/*` : integraciones externas (Stripe, ImageKit) y navegacion.
7. `lib/validators/*` : validaciones Zod para auth y checkout.
8. `prisma/schema.prisma` : modelo de datos completo.
9. `prisma/seed.ts` : datos demo funcionales.
10. `lib/db.ts` : cliente Prisma singleton.
11. `lib/env.ts` : validacion de variables de entorno.
12. `proxy.ts` : proteccion de rutas privadas a nivel edge/proxy.

Observaciones de estructura:

1. La app esta organizada por dominios funcionales, no por capas monoliticas.
2. Se usan Server Components por defecto; los Client Components se reservan para modales, formularios, drawers, filtros interactivos y navegacion por teclado.
3. Hay archivos "compatibilidad" en `components/shop/*` y `lib/repositories/shop.ts` que delegan en la implementacion actual de tienda ( `store` ).

---

### 3. Funcionamiento general de la aplicacion (recorrido de usuario)

---

Recorrido tipico:

1. El usuario entra en `/` y ve hero, conciertos proximos, productos destacados y noticias.
2. Desde el header puede abrir buscador global, navegar a secciones y abrir carrito/perfil.
3. En contenido publico puede explorar:
  - conciertos futuros/pasados con filtros,
  - noticias y bio de banda,
  - catalogo de musica con modales,
  - media de fotos y videos.
1. En tienda ( `/store` ) puede filtrar, ordenar, paginar y abrir detalle de producto.
2. Para anadir al carrito necesita sesion.
3. En carrito ( `/carrito` ) puede actualizar cantidad, eliminar items o ir a checkout.
4. En checkout ( `/checkout` ) rellena envio/facturacion, elige metodo de pago, y si elige tarjeta se crea sesion Stripe.
5. Stripe procesa el pago y redirige a `/checkout/success?session_id=...`.
6. La app sincroniza estado del pedido contra Stripe y muestra resultado final.
7. El usuario autenticado ve su pagina `/account` con perfil y accesos rapidos.

---

### 4. Explicacion detallada de cada pagina (rutas reales)

---

`/` (home)

1. Archivo: `app/page.tsx`.

2. Objetivo: landing principal.

3. Muestra:

- hero,
- proximos conciertos,
- productos destacados,
- ultimas noticias.

1. Datos: `getHomeSnapshot()` desde `lib/repositories/site.ts`.

2. Componentes principales: `PageHero`, `EmptyState`,  
`components/shop/product-card`.

3. Acciones de usuario: navegar a conciertos, tienda y banda.

### **/concerts**

1. Archivo: `app/concerts/page.tsx`.

2. Objetivo: ruta raiz de conciertos.

3. Comportamiento: redirige a `/concerts/upcoming`.

### **/concerts/upcoming**

1. Archivo: `app/concerts/upcoming/page.tsx`.

2. Objetivo: conciertos futuros.

3. Datos: `getConcertCatalog("upcoming", searchParams)`.

4. Componentes: `ConcertsCatalogPage` (usa filtros, cards, paginacion y modal info).

5. Acciones: filtrar por fecha/continente/pais, paginar, abrir modal, ir a ticket o gratis.

### **/concerts/past**

1. Archivo: `app/concerts/past/page.tsx`.

2. Objetivo: historico de conciertos.

3. Datos: `getConcertCatalog("past", searchParams)`.

4. Componentes: mismos bloques que upcoming, con detalle extra (cronica, tracklist, links, fotos/videos).

5. Acciones: filtrar, paginar y abrir modal de detalle.

**/band**

1. Archivo: `app/band/page.tsx` .
2. Objetivo: raiz de seccion banda.
3. Comportamiento: redirige a `/band/news` .

**/band/news**

1. Archivo: `app/band/news/page.tsx` .
2. Objetivo: listado de noticias.
3. Datos: `getBandNewsCatalog(searchParams)` .
4. Componentes: `BandNewsFilters` , `BandNewsListClient` , `BandNewsPagination` .
5. Acciones: filtrar por fechas/tag, paginar, expandir/colapsar contenido de noticia.

**/band/bio**

1. Archivo: `app/band/bio/page.tsx` .
2. Objetivo: hub de bio.
3. Muestra: accesos a biografia y miembros.
4. Componentes: `PageHero` y tarjetas de navegacion.

**/band/bio/biography**

1. Archivo: `app/band/bio/biography/page.tsx` .
2. Objetivo: biografia por secciones con anclas.
3. Datos: `getBandBiographySections()` .
4. Componentes: `BiographySections` , `EmptyState` .
5. Acciones: navegar por indice lateral a secciones.

**/band/bio/members**

1. Archivo: `app/band/bio/members/page.tsx` .
2. Objetivo: directorio de miembros y colaboradores.
3. Datos: `getBandMembersDirectory()` .
4. Componentes: `MembersDirectoryClient` , `EmptyState` .
5. Acciones: abrir modal de detalle por persona.

## **/musica**

1. Archivo: `app/musica/page.tsx` .
2. Objetivo: catalogo mixto canciones + albumes.
3. Datos: `getMusicCatalog(searchParams)` .
4. Componentes: `MusicFilters` , `MusicCatalogClient` , `MusicPagination` .
5. Acciones: filtrar, ordenar, paginar, abrir modal de cancion o album, y desde album abrir track (cancion).

## **/media**

1. Archivo: `app/media/page.tsx` .
2. Objetivo: hub de media.
3. Datos: `getMediaOverviewStats()` .
4. Muestra: accesos a fotos y videos con conteos.

## **/media/photos**

1. Archivo: `app/media/photos/page.tsx` .
2. Objetivo: listado de albumes de fotos.
3. Datos: `getPhotoAlbumsCatalog(searchParams)` .
4. Componentes: `PhotoFilters` , `PhotoPagination` , `EmptyState` .
5. Acciones: filtrar, ordenar, paginar, abrir detalle de album.

## **/media/photos/[slug]**

1. Archivo: `app/media/photos/[slug]/page.tsx` .
2. Objetivo: detalle de album.
3. Datos: `getPhotoAlbumDetailBySlug(slug)` .
4. Componentes: `PhotoGalleryClient` .
5. Acciones: cambiar miniatura, navegar foto anterior/siguiente.

## **/media/videos**

1. Archivo: `app/media/videos/page.tsx` .
2. Objetivo: listado de colecciones o videos unicos.
3. Datos: `getVideoCatalog(searchParams)` .
4. Componentes: `VideoFilters` , `VideoPagination` , `EmptyState` .

5. Acciones: filtrar, ordenar, paginar, abrir detalle.

### **/media/videos/[slug]**

1. Archivo: `app/media/videos/[slug]/page.tsx` .

2. Objetivo: detalle de coleccion o video unico.

3. Datos: `getVideoDetailBySlug(slug)` .

4. Componentes:

- coleccion: `VideoCollectionView` ,
- single: iframe embebido directo.

1. Acciones: seleccionar video de coleccion o navegar de vuelta.

### **/fanclub**

1. Archivo: `app/fanclub/page.tsx` .

2. Objetivo: placeholder elegante.

3. Muestra: mensaje de "proximamente" y descripcion de la idea.

### **/store**

1. Archivo: `app/store/page.tsx` .

2. Objetivo: catalogo de tienda con filtros avanzados.

3. Datos: `getStoreCatalog(searchParams)` .

4. Componentes: `StoreFiltersSidebar` , `StoreProductCard` ,  
`StorePagination` , `EmptyState` .

5. Acciones: filtrar por categoria/subcategoria/precio/talla/genero/tipo media, ordenar y paginar.

### **/store/[slug]**

1. Archivo: `app/store/[slug]/page.tsx` .

2. Objetivo: detalle de producto.

3. Datos:

- `getStoreProductBySlug(slug)` ,
- `getRelatedStoreProducts(...)` ,
- `isPhysicalMediaWithNotes(...)` .

1. Muestra: galeria, precio, descripcion, selector de cantidad/talla, tracklist/liner notes para media fisica.

2. Accion clave: anadir al carrito ( `StoreProductPurchaseForm` ).

### **/buscar**

1. Archivo: `app/buscar/page.tsx` .
2. Objetivo: resultados completos de busqueda global.
3. Datos: `searchHeaderContent(query, { limit: 24 })` .
4. Muestra: coincidencias en paginas y productos.

### **/login**

1. Archivo: `app/login/page.tsx` .
2. Objetivo: inicio de sesion.
3. Datos: comprueba sesion con `getSessionUser()` .
4. Componentes: `LoginForm` .
5. Accion: autenticarse y volver a landing.

### **/registro**

1. Archivo: `app/registro/page.tsx` .
2. Objetivo: crear cuenta.
3. Datos: comprueba sesion con `getSessionUser()` .
4. Componentes: `RegisterForm` .
5. Accion: registrarse y volver a landing.

### **/account (privada)**

1. Archivo: `app/account/page.tsx` .
2. Objetivo: resumen de perfil.
3. Proteccion: `requireSession("/account")` .
4. Datos: `getCurrentUser()` .
5. Muestra: datos de usuario y accesos rapidos a carrito/tienda.

### **/carrito (privada)**

1. Archivo: `app/carrito/page.tsx` .
2. Objetivo: carrito completo.
3. Proteccion: `requireSession("/carrito")` .
4. Datos: `getCartForUser(userId)` .



5. Componentes: `CartItem` .

6. Acciones: actualizar cantidad, quitar item, vaciar carrito, ir a checkout.

### **/checkout (privada)**

1. Archivo: `app/checkout/page.tsx` .

2. Objetivo: envio + facturacion + pago.

3. Proteccion: `requireSession("/checkout")` .

4. Datos:

- carrito: `getCartForUser` ,
- prefill: `getCheckoutPrefill` .

1. Componente principal: `CheckoutFlow` .

2. Acciones: validar formulario y crear sesion de pago Stripe.

### **/checkout/success (privada)**

1. Archivo: `app/checkout/success/page.tsx` .

2. Objetivo: resultado final de pago.

3. Proteccion: `requireSession("/checkout/success")` .

4. Datos:

- sincroniza estado:  
`syncOrderWithStripeCheckoutSession(session_id)` ,
- resumen: `getOrderSummaryBySession(...)` .

1. Acciones: volver a cuenta o tienda/reintento.

## **APIs internas**

1. `app/api/search/route.ts` : busqueda rapida (paginas + productos).
2. `app/api/checkout/route.ts` : valida checkout, guarda direcciones, crea orden pendiente y sesion Stripe.
3. `app/api/stripe/webhook/route.ts` : actualiza estado de orden segun eventos Stripe.
4. `app/api/imagekit/auth/route.ts` : firma de autenticacion ImageKit (preparada para subidas autenticadas).

---

## 5. Explicacion detallada de componentes importantes

---

### Layout y navegacion

1. `components/layout/site-header.tsx` : componente server que prepara datos de usuario y carrito para el header.
2. `components/layout/header-client.tsx` : header interactivo (menu desktop/mobile, perfil, auth modal, drawer de carrito).
3. `components/layout/global-search.tsx` : buscador con debounce, resultados rapidos y navegacion por teclado.
4. `components/layout/site-footer.tsx` : footer con enlaces principales.

### Auth

1. `components/auth/auth-modal.tsx` : modal accesible con cierre por ESC y foco.
2. `components/auth/auth-modal-panel.tsx` : conmutador login/registro dentro del modal.
3. `components/auth/login-form.tsx` : formulario login conectado a server action.
4. `components/auth/register-form.tsx` : formulario de alta completo conectado a server action.
5. `components/auth/auth-submit-button.tsx` : boton submit con estado `pending`.

### Tienda

1. `components/store/store-filters-sidebar.tsx` : filtros de tienda sincronizados por query string.
2. `components/store/store-product-card.tsx` : tarjeta de producto reusable.
3. `components/store/store-pagination.tsx` : paginador que conserva filtros en URL.
4. `components/store/store-product-purchase-form.tsx` : selector de talla/cantidad y alta en carrito.

## Carrito y checkout

1. `components/cart/cart-drawer.tsx` : drawer lateral del carrito (desde header).
2. `components/cart/cart-line-item.tsx` : fila editable del carrito completo.
3. `components/checkout/checkout-flow.tsx` : flujo completo checkout (estado local + validacion + llamada API).
4. `components/checkout/checkout-address-fields.tsx` : bloque de campos envio/facturacion reutilizable.

## Conciertos

1. `components/concerts/concerts-catalog-page.tsx` : pagina base que compone hero/filtros/listado/paginacion.
2. `components/concerts/concert-cards-client.tsx` : tarjetas y modal de detalle.
3. `components/concerts/concert-filters.tsx` : formulario de filtros.
4. `components/concerts/concert-pagination.tsx` : paginacion conservando filtros.

## Banda

1. `components/band/news-list-client.tsx` : noticias expandibles con enlaces relacionados.
2. `components/band/news-filters.tsx` : filtros de fecha y tag.
3. `components/band/news-pagination.tsx` : paginador de noticias.
4. `components/band/biography-sections.tsx` : biografia por secciones con indice lateral y anclas.
5. `components/band/members-directory-client.tsx` : directorio con modal de detalle de persona.

## Musica

1. `components/music/music-catalog-client.tsx` : listado y modales de cancion/album; tracklist de album enlaza a modal cancion.
2. `components/music/music-filters.tsx` : filtros por fecha, tipo y orden.
3. `components/music/music-pagination.tsx` : paginacion.

## Media

1. `components/media/photo-gallery-client.tsx` : miniaturas + foto grande + navegacion lateral/teclas.
2. `components/media/video-collection-viewer.tsx` : selector de videos y reproductor embebido.
3. `components/media/photo-filters.tsx` , `video-filters.tsx` : filtros.
4. `components/media/photo-pagination.tsx` , `video-pagination.tsx` : paginacion.

## UI base

1. `components/ui/page-hero.tsx` : cabecera visual consistente entre secciones.
2. `components/ui/empty-state.tsx` : estado vacio.
3. `components/ui/loading-grid.tsx` : skeleton loading reutilizable.

---

## 6. Funciones y metodos importantes (archivo, tipo y rol)

---

### Autenticacion y sesion

---

1. `registerAction`

Archivo: `lib/auth/actions.ts` Tipo: server action Parametros: estado previo + `FormData` de registro Funcion: valida con Zod, hashlea password con bcrypt, crea usuario+carrito, crea sesion, dirige `/`. Importancia: punto de entrada principal de alta de usuario.

1. `loginAction`

Archivo: `lib/auth/actions.ts` Tipo: server action Parametros: estado previo + `FormData` login Funcion: valida, busca usuario, compara hash, crea sesion, dirige `/`. Importancia: login persistente real.

1. `logoutAction`

Archivo: `lib/auth/actions.ts` Tipo: server action Funcion: limpia cookie de sesion y dirige a home.

1. `requireSession`

Archivo: `lib/auth/dal.ts` Tipo: helper server Parametro: ruta de retorno  
Funcion: si no hay sesion, redirige a `/login?redirect=...`; si hay, devuelve usuario de sesion.

1. `createSessionToken`, `decodeSessionToken`, `setSession`,  
`clearSession`

Archivo: `lib/auth/session.ts` Tipo: servicio auth Funcion: construccion y verificacion JWT (jose), y gestion de cookie `sb_session`.

1. `proxy`

Archivo: `proxy.ts` Tipo: edge/proxy handler Funcion: protege rutas privadas por presencia de cookie antes de llegar a pagina.

## Carrito

---

1. `addToCartAction`, `updateCartItemAction`,  
`removeCartItemAction`, `clearCartAction`

Archivo: `lib/cart/actions.ts` Tipo: server actions Funcion: validan inputs con Zod, obligan sesion y llaman repositorio de carrito.

1. `addItemToCart`

Archivo: `lib/repositories/cart.ts` Tipo: repositorio Prisma Parametros: `userId`, `productId`, `productVariantId?`, `quantity?` Funcion: crea/asegura carrito, valida stock y variante, inserta o suma cantidad en `CartItem`.

1. `getCartForUser`

Archivo: `lib/repositories/cart.ts` Tipo: repositorio Prisma Funcion: lee carrito con relaciones y devuelve vista calculada ( `totalItems`, `subtotal` ).

1. `updateCartItemQuantity`, `removeCartItem`, `clearCartForUser`

Archivo: `lib/repositories/cart.ts` Funcion: operaciones de mantenimiento de carrito.

## Checkout y direcciones

---

### 1. `getCheckoutPrefill`

Archivo: `lib/repositories/checkout.ts` Tipo: repositorio Prisma

Funcion: carga direcciones default de envio/facturacion y construye datos iniciales del formulario.

### 1. `saveCheckoutAddresses`

Archivo: `lib/repositories/checkout.ts` Tipo: repositorio Prisma (write)

Funcion: guarda/actualiza direcciones default y sincroniza pais/telefono en usuario.

### 1. `POST /api/checkout`

Archivo: `app/api/checkout/route.ts` Tipo: route handler server Funcion:

- valida payload checkout,
- guarda direcciones,
- exige `paymentMethod=card`,
- crea pedido pendiente,
- crea sesion Stripe Checkout,
- guarda `checkoutSessionId` y `paymentIntentId`,
- devuelve URL de Stripe.

## Pedidos y Stripe

---

### 1. `createPendingOrderFromCart`

Archivo: `lib/repositories/orders.ts` Tipo: repositorio Prisma (write)

Funcion: crea pedido `PENDING` con snapshots de item y de direcciones.

### 1. `attachStripeCheckoutSessionToOrder`

Archivo: `lib/repositories/orders.ts` Funcion: enlaza IDs de Stripe al pedido.

### 1. `markOrderAsPaid`, `markOrderAsFailed`, `markOrderAsFailedByPaymentIntent`

Archivo: `lib/repositories/orders.ts` Funcion: transiciones de estado de pago/orden y limpieza de carrito tras pago correcto.

1. `syncOrderWithStripeCheckoutSession`

Archivo: `lib/repositories/orders.ts` Funcion: consulta Stripe por `session_id` y sincroniza estado en DB.

1. `POST /api/stripe/webhook`

Archivo: `app/api/stripe/webhook/route.ts` Funcion: verifica firma y procesa eventos `checkout.session.completed`, `...failed`, `...expired`, `payment_intent.payment_failed`.

## Busqueda

---

1. `searchHeaderContent`

Archivo: `lib/repositories/search.ts` Tipo: repositorio mixto (paginas estaticas + Prisma productos) Funcion: devuelve resultados para header y pagina `/buscar`.

1. `searchSitePages`

Archivo: `lib/search/pages.ts` Tipo: helper de scoring local Funcion: ranking de paginas internas segun termino de busqueda.

## Catalogos de contenido

---

1. `getStoreCatalog`, `getStoreProductBySlug`

Archivo: `lib/repositories/store.ts` Funcion: construyen catalogo de tienda con filtros/orden/paginacion y detalle.

1. `getConcertCatalog`

Archivo: `lib/repositories/concerts.ts` Funcion: separa upcoming/past, aplica filtros de ubicacion/fechas, enriquece detalle de venue/media.

1. `getBandNewsCatalog`, `getBandBiographySections`, `getBandMembersDirectory`

Archivo: `lib/repositories/band.ts` Funcion: alimentan noticias, biografia y miembros.

1. `getMusicCatalog`

Archivo: `lib/repositories/music.ts` Funcion: mezcla canciones y albums, arma diccionarios de detalle para modales.

1. `getPhotoAlbumsCatalog` , `getPhotoAlbumDetailBySlug` ,  
`getVideoCatalog` , `getVideoDetailBySlug`

Archivo: `lib/repositories/media.ts` Funcion: catalogos y detalles de media con filtros/paginacion.

## Integracion de imagen y utilidades

---

1. `resolveImageUrl`

Archivo: `lib/services/imagekit.ts` Tipo: helper URL Funcion: transforma path relativo a endpoint ImageKit o devuelve URL absoluta si ya viene completa.

1. `getImageKitClient`

Archivo: `lib/services/imagekit-server.ts` Tipo: servicio server  
Funcion: inicializa cliente ImageKit para auth de subida.

1. `withDatabaseFallback`

Archivo: `lib/repositories/safe-query.ts` Funcion: captura ciertos errores Prisma recuperables y devuelve fallback.

1. `formatCurrency` , `formatDate`

Archivo: `lib/utlis.ts` Funcion: formato coherente de moneda y fecha para la UI.

## Nota de uso real de funciones

---

1. `persistOrderFromCart` existe en `lib/repositories/cart.ts` pero no se usa en el flujo actual (el flujo activo usa `lib/repositories/orders.ts` ).
2. `getCartCount` existe pero no es la via usada en header actual (header carga carrito completo y de ahi calcula count).

---



## 7. Funcionamiento de ImageKit en este proyecto

---

Que es ImageKit aqui:

1. Es la capa de entrega de imagenes y assets multimedia en frontend.
2. Se usa para productos, noticias, banda, musica, fotos, videos y detalles de conciertos (algunas URLs hardcoded en contenido extra).

Como esta configurado:

1. Variable publica: `NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT` .
2. Variables server: `IMAGEKIT_PUBLIC_KEY` , `IMAGEKIT_PRIVATE_KEY` .
3. Archivo de entorno validado en `lib/env.ts` .
4. `next.config.ts` agrega host remoto de ImageKit a `images.remotePatterns` .

Piezas de integracion:

1. `lib/services/imagekit.ts` : `resolveImageUrl()` (cliente/servidor) para construir URL final.
2. `lib/services/imagekit-server.ts` : cliente ImageKit server-side.
3. `app/api/imagekit/auth/route.ts` : endpoint que entrega parametros firmados de autenticacion.

Donde se usa en codigo:

1. Tienda: `components/store/store-product-card.tsx` , `app/store/[slug]/page.tsx` , `components/store/store-product-purchase-form.tsx` (indirectamente por imagenes de producto).
2. Carrito/checkout: `components/cart/*` , `components/checkout/checkout-flow.tsx` .
3. Banda: `components/band/*` para news, bio y miembros.
4. Musica: `components/music/music-catalog-client.tsx` .
5. Media: `app/media/photos/page.tsx` , `app/media/videos/page.tsx` , `components/media/photo-gallery-client.tsx` .
6. Buscador: `components/layout/global-search.tsx` .

Estado real de la integracion ImageKit:

1. La carga de imagenes esta funcional (resolver URLs y mostrarlas).

2. Existe endpoint de auth para upload ( `/api/imagekit/auth` ), pero no hay UI de subida ni flujo CMS conectado en este repositorio actual.

---

## 8. Funcionamiento de Stripe en este proyecto

---

Papel de Stripe:

1. Metodo real de pago con tarjeta.
2. Se usa Stripe Checkout hospedado (no Stripe Elements en frontend).

Archivos clave:

1. `lib/services/stripe.ts` : inicializa cliente Stripe con `STRIPE_SECRET_KEY` .
2. `app/api/checkout/route.ts` : crea sesion Checkout.
3. `app/api/stripe/webhook/route.ts` : procesa eventos asincornos.
4. `lib/repositories/orders.ts` : crea y actualiza pedidos segun estado de pago.
5. `app/checkout/success/page.tsx` : sincroniza estado al volver de Stripe.

Flujo de pago real:

1. Usuario completa `CheckoutFlow` en `/checkout` .
2. Frontend envia JSON a `POST /api/checkout` .
3. Backend valida payload ( `checkoutPayloadSchema` ).
4. Guarda direcciones por `saveCheckoutAddresses` .
5. Crea pedido pendiente con `createPendingOrderFromCart` .
6. Crea sesion Stripe Checkout con line items reales.
7. Guarda `checkoutSessionId` / `paymentIntentId` en pedido.
8. Frontend redirige al `session.url` de Stripe.
9. Stripe redirige a `/checkout/success?session_id=...` .
10. Pagina success ejecuta `syncOrderWithStripeCheckoutSession` .
11. Si pagado, pedido pasa a `PAID/PROCESSING` y se vacia carrito.
12. Webhook puede confirmar o corregir estado tambien en paralelo.

Eventos webhook implementados:

1. `checkout.session.completed` -> pago confirmado.
2. `checkout.session.async_payment_failed` y `checkout.session.expired` -> pago fallido.
3. `payment_intent.payment_failed` -> marca fallo por intent.

PayPal:

1. Solo UI en `CheckoutFlow` (radio button "proximamente").
2. El backend rechaza `paymentMethod !== "card"` con mensaje claro.

Notas reales:

1. `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` esta en env schema, pero no se usa porque el flujo actual no monta Stripe Elements client-side.
2. Hay estructura preparada para webhooks (endpoint y logica), no solo redirect success.

---

## 9. Base de datos y Prisma (implementacion real)

---

Rol de la base de datos:

1. Guarda usuarios, auth, direcciones, catalogo tienda, carrito, pedidos, conciertos, noticias, bio, musica, fotos y videos.
2. Todo el contenido dinamico visible en UI sale de esta DB (excepto algunas listas estaticas de busqueda/navegacion y contenido extra hardcoded).

Que es Prisma aqui:

1. Es el ORM que define el modelo de datos y ejecuta consultas/escrituras.
2. El schema esta en `prisma/schema.prisma`.
3. El cliente se genera en `app/generated/prisma`.

Configuracion Prisma/Neon:

1. Cliente runtime: `lib/db.ts` usa `PrismaClient` + `PrismaPg` adapter con `DATABASE_URL`.
2. CLI Prisma: `prisma.config.ts` usa `DIRECT_URL` para comandos/migraciones.

3. Validacion env: `lib/env.ts`.

Modelos principales del schema:

1. Usuarios y auth: `User`, `Address`.
2. Tienda: `ProductCategory`, `Product`, `ProductImage`, `ProductVariant`.
3. Carrito/pedido: `Cart`, `CartItem`, `Order`, `OrderItem`.
4. Conciertos y banda: `Concert`, `News`, `BandMember`, `BiographySection`.
5. Musica: `MusicRelease`, `Track`, `MusicContributor`, `ReleaseContributor`, `TrackContributor`.
6. Media: `PhotoAlbum`, `Photo`, `VideoCollection`, `VideoItem`.
7. Enums de negocio: roles de usuario, estados de pedido/pago, tipos de producto, tipos de release/track, etc.

Relaciones importantes:

1. `User` 1-1 `Cart`; `User` 1-N `Order`; `User` 1-N `Address`.
2. `ProductCategory` arbol (parent/children) + `Product`.
3. `Product` 1-N `ProductImage` y 1-N `ProductVariant`.
4. `Cart` 1-N `CartItem`; `CartItem` apunta a `Product` y `ProductVariant`.
5. `Order` 1-N `OrderItem`; snapshots guardan datos historicos aunque cambie el catalogo.
6. `MusicRelease` 1-N `Track`; contributors via tablas intermedias `ReleaseContributor` y `TrackContributor`.
7. `PhotoAlbum` 1-N `Photo`; `VideoCollection` 1-N `VideoItem`.

Como se consulta Prisma en la app:

1. Lecturas centralizadas en `lib/repositories/*`.
2. Escrituras en:
  - auth actions,
  - cart actions/repository,
  - checkout/orders repositories,
  - seed.

Secciones de app y uso Prisma:

1. Productos/tienda: `lib/repositories/store.ts` .
2. Conciertos: `lib/repositories/concerts.ts` .
3. Noticias/banda: `lib/repositories/band.ts` .
4. Musica: `lib/repositories/music.ts` .
5. Media: `lib/repositories/media.ts` .
6. Usuarios/sesion/perfil: `lib/auth/dal.ts` , `lib/auth/actions.ts` .
7. Carrito: `lib/repositories/cart.ts` , `lib/cart/actions.ts` .
8. Pedidos/pago: `lib/repositories/orders.ts` ,  
`app/api/checkout/route.ts` , `app/api/stripe/webhook/route.ts` .

Snapshots de direccion en pedidos:

1. `Order` guarda campos `shipping*` y `billing*` .
2. Esto evita perder el contexto del pedido aunque el usuario cambie direcciones despues.

Seed de datos:

1. Archivo: `prisma/seed.ts` .
2. Crea datos demo funcionales minimos: 1 user, categorias, 1 producto ropa, conciertos, 1 noticia, 1 album+1 track, 1 album de fotos+1 foto.
3. Tambien crea contributors y credits musicales.

Migraciones:

1. Hay script `npm run prisma:migrate` .
2. En este estado del repo no existe carpeta `prisma/migrations` (no hay migraciones versionadas en disco).
3. La configuracion y scripts existen, pero la historia de migraciones no esta incluida actualmente en el repo.

Manejo de errores de DB:

1. `withDatabaseFallback` ( `lib/repositories/safe-query.ts` )  
captura errores Prisma recuperables concretos ( `P1001` , `P2021` ,  
`P2022` ) y devuelve fallback.
2. Esto evita caidas completas en algunas lecturas cuando hay problemas temporales o de schema.

---

## 10. Autenticacion y cuenta de usuario

---

Registro:

1. UI: `components/auth/register-form.tsx`.
2. Validacion cliente/servidor via Zod ( `lib/validators/auth.ts` ).
3. Server action `registerAction` :
  - valida,
  - hash de password con bcrypt,
  - crea usuario,
  - crea carrito inicial,
  - abre sesion.

Login:

1. UI: `components/auth/login-form.tsx`.
2. `loginAction` valida credenciales y compara hash.
3. Si es correcto crea cookie de sesion.

Sesion persistente:

1. JWT firmado con `SESSION_SECRET` ( `jose` ) en `lib/auth/session.ts`.
2. Cookie `sb_session` con `httpOnly`, `sameSite=lax`, `secure` en prod.
3. Opcion remember extiende duracion de sesion.

Proteccion de rutas:

1. Primera barrera: `proxy.ts`.
2. Segunda barrera: chequeo server `requireSession` dentro de paginas/actions.

Header autenticado/no autenticado:

1. `SiteHeader` y `HeaderClient` muestran submenu perfil.
2. No autenticado: opciones Login/Registro.
3. Autenticado: "Hola + nombre", "Cuenta", "Cerrar sesion".

Pagina de cuenta:

1. `/account` muestra datos de perfil y accesos rapidos.

2. No hay por ahora editor de perfil ni historico de pedidos en UI.

---

## 11. Tienda, carrito y checkout (e-commerce)

---

Catalogo de tienda:

1. Ruta `/store`.
2. Filtros por query params (categoria, subcategoria, precio, talla, genero, tipo media, orden, pagina).
3. Backend parsea y valida filtros con `parseStoreFilters`.

Detalle de producto:

1. Ruta `/store/[slug]`.
2. Muestra imagen principal, miniaturas, precio, descripcion, selector de talla si aplica.
3. Si es media fisica ( `cd` / `vinilo` ) muestra tracklist y liner notes desde metadata.

Añadir al carrito:

1. `StoreProductPurchaseForm` envia a `addToCartAction`.
2. El backend valida y usa `addItemToCart`.
3. Si no hay sesion, redirige a login.

Carrito:

1. Drawer desde header ( `CartDrawer` ) y pagina completa ( `/carrito` ).
2. Permite eliminar, actualizar cantidad, vaciar carrito.
3. Totales se calculan a partir de snapshots de precio en `CartItem`.

Checkout:

1. Ruta protegida `/checkout`.
2. Formulario de envio y facturacion (con checkbox "usar misma direccion").
3. Validacion en cliente ( `checkoutPayloadSchema` ) y en servidor ( `/api/checkout` ).
4. Borrador persistente en `localStorage` durante el flujo ( `sugarbay.checkout.draft.v1` ).

5. Boton de continuar depende de validacion correcta y metodo tarjeta.

Pago:

1. Metodo real: tarjeta con Stripe.
2. PayPal: solo placeholder visual.

---

## 12. Conciertos, banda, musica y media (funcionamiento por seccion)

---

### Conciertos

---

1. Muestra proximos y pasados separados por fecha.
2. Fuente de datos: `lib/repositories/concerts.ts` sobre modelo `Concert`.
3. Filtros: rango de fechas, continente, pais.
4. Cards y modal de detalle con:
  - venue,
  - maps,
  - experiencias,
  - links,
  - cronica/tracklist/media para pasados.
1. `lib/concerts/content.ts` inyecta detalles extra hardcoded por slug.

### Banda (noticias + bio)

---

1. Noticias:
  - lista por fecha desc,
  - expandir/colapsar contenido,
  - filtros fecha/tag + paginacion.
1. Bio:
  - `biography` : secciones con indice y anclas.
  - `members` : separa banda y colaboradores, con modal de detalle.



1. Fuente de datos: `lib/repositories/band.ts` y modelos `News`, `BiographySection`, `BandMember`.

## Musica

---

1. Catalogo mixto song/album con filtros de fecha/tipo/orden.
2. Cada item abre modal detalle.
3. En modal album, cada track abre modal de cancion (flujo album -> track -> song funcional en UI).
4. Fuente de datos: `lib/repositories/music.ts` y modelos `MusicRelease`, `Track`, `contributors`.

## Media

---

1. Fotos:
  - listado de albumes con filtros,
  - detalle con miniaturas, foto principal y navegacion.
1. Videos:
  - listado de colecciones o videos unicos,
  - detalle con iframes embebidos (YouTube/Vimeo transformados).
1. Fuente: `lib/repositories/media.ts` con modelos `PhotoAlbum/Photo` y `VideoCollection/VideoItem`.

---

## 13. Flujo tecnico resumido (conexion entre capas)

---

Flujo tecnico de extremo a extremo:

1. Usuario interactua con UI en `app/*` y `components/*`.
2. Si la vista es server-side, la pagina llama repositorios en `lib/repositories/*`.
3. Repositorios usan `prisma` (`lib/db.ts`) para leer/escribir en Neon PostgreSQL.

4. Para acciones mutables (auth/cart), Client Components envian formularios a server actions.
5. Para checkout y webhooks, se usan route handlers ( `app/api/*` ).
6. Stripe:
  - `api/checkout` crea sesion y guarda pedido pendiente,
  - `api/stripe/webhook` confirma/cancela estado,
  - `/checkout/success` sincroniza estado al volver.
1. ImageKit:
  - `resolveImageUrl` construye URL final para `next/image` ,
  - endpoint de auth listo para futuras subidas.

---

## 14. Puntos fuertes del proyecto

---

1. Arquitectura modular por dominio ( `repositories` , `components` por seccion).
2. Uso coherente de App Router con Server Components por defecto.
3. TypeScript estricto y Zod en validaciones clave.
4. Flujo e-commerce completo: producto -> carrito -> checkout -> Stripe -> confirmacion.
5. Sistema de estados `loading` , `empty` y `error` muy extendido en rutas.
6. Header avanzado con buscador global, perfil y carrito drawer.
7. Modelo Prisma amplio y escalable, con snapshots de pedido bien planteados.
8. Integracion Stripe real (no mock) con webhook.
9. UI consistente mediante componentes base ( `PageHero` , `EmptyState` , `LoadingGrid` ) y design tokens en `globals.css` .

---

## 15. Limitaciones o partes mejorables (estado actual real)

---

1. No hay tests automaticos (unitarios/integracion/e2e) en el repo actual.
2. No existe `.env.example` en disco actualmente, aunque `README.md` lo menciona.
3. No hay carpeta `prisma/migrations` incluida; el historial de migraciones no esta versionado aqui.
4. Existen algunos textos con problemas de codificacion ( `Â•` , acentos perdidos) en strings de UI.
5. `persistOrderFromCart` en `lib/repositories/cart.ts` no se usa en el flujo actual.
6. Endpoint `GET /api/imagekit/auth` esta preparado, pero no hay UI de subida de archivos conectada.
7. `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` se valida en env pero no se utiliza en runtime actual (flujo usa Checkout hospedado).
8. `proxy.ts` protege tambien `/cuenta` , pero esa ruta no existe como pagina en `app/` .
9. No hay panel admin/CMS para gestionar contenido; todo depende de DB/seed.
10. Cuenta de usuario no incluye aun gestion de pedidos historicos ni edicion avanzada de perfil.

---

## 16. Conclusion final

---

Sugarbay, en su estado actual, es una aplicacion full-stack funcional y coherente: combina contenido musical/publico con e-commerce real y pago Stripe, sobre una base de datos bien modelada con Prisma. La estructura del codigo esta pensada para crecer por dominios, y las piezas clave (auth, carrito, checkout, catalogos y media) estan conectadas de forma clara entre frontend, backend y servicios externos.

No es una demo vacia: hay flujo completo de compra y contenido dinamico real. A la vez, todavia hay margen de mejora para nivel produccion total (tests, migraciones versionadas, panel de gestion y cierre de detalles de UX/texto).

---

## 17. Resumen especifico de Prisma para exposicion (oral)

---

Prisma es la capa que conecta el codigo con la base de datos de forma tipada y segura. En este proyecto:

1. El modelo de datos completo esta en `prisma/schema.prisma`.
2. Desde `lib/repositories/*` se hacen todas las consultas y escrituras.
3. Prisma se conecta a Neon PostgreSQL usando `DATABASE_URL` y el adapter `PrismaPg`.
4. Interviene en todas las secciones: tienda, conciertos, noticias, musica, media, usuarios, carrito y pedidos.
5. Es clave porque evita SQL manual repetitivo, mantiene tipado con TypeScript y hace el proyecto mas mantenible y escalable.

---

## 18. Resumen especifico de funciones/metodos clave para exposicion (oral)

---

Funciones clave para contar en una exposicion:

1. `registerAction` y `loginAction` (`lib/auth/actions.ts`): altas y acceso de usuario con validacion + hash + sesion.
2. `requireSession` (`lib/auth/dal.ts`): protege rutas privadas.
3. `addToCartAction` + `addItemToCart` (`lib/cart/actions.ts`, `lib/repositories/cart.ts`): anadir al carrito con control de stock.
4. `getStoreCatalog` (`lib/repositories/store.ts`): catalogo tienda con filtros avanzados.
5. `POST /api/checkout` (`app/api/checkout/route.ts`): valida checkout y crea sesion Stripe.
6. `createPendingOrderFromCart` / `markOrderAsPaid` (`lib/repositories/orders.ts`): ciclo de vida del pedido.
7. `POST /api/stripe/webhook` (`app/api/stripe/webhook/route.ts`): sincroniza estado de pago.

8. `resolveImageUrl` ( `lib/services/imagekit.ts` ): sirve imagenes de forma consistente desde ImageKit.
9. `getConcertCatalog` , `getBandNewsCatalog` , `getMusicCatalog` , `getPhotoAlbumsCatalog` (repositorios): alimentan toda la capa de contenido.

Estas funciones son importantes porque conectan la interfaz con la logica de negocio y la base de datos, y hacen que la app funcione de extremo a extremo.

---

## Anexo breve: comprobaciones realizadas para este informe

---

1. Se revisaron rutas en `app/` , componentes en `components/` , logica en `lib/` , schema/seed en `prisma/` .
2. Se revisaron integraciones de Stripe e ImageKit en rutas API y servicios.
3. Se valido estado de lint ( `npm.cmd run lint` ) sin errores en el entorno local.
4. Este informe evita suposiciones y se basa en implementacion existente en el repositorio actual.